
Kubernetes Services: Interview Questions & Answers (Production-Level)

1. What is a Kubernetes Service and why is it needed in production?

Answer:

A Service is an abstraction that defines a stable network identity (DNS name + IP) for accessing a set of pods. It:

- Decouples service consumers from dynamic pod IPs
- Enables **load balancing** across pods
- Works with **Service Discovery** ([kube-dns](#))
- Supports **exposing workloads** internally or externally

2. Explain the differences between ClusterIP, NodePort, and LoadBalancer services.

Type	Exposed To	Port Mapping	Use Case
ClusterIP	Internal cluster	ClusterIP:Port → Pod:TargetPort	Default, internal services
NodePort	External + internal	NodeIP:NodePort → ClusterIP:Port → Pod	Dev/testing, direct node access
LoadBalancer	External (via cloud LB)	ExternalIP:Port → NodePort → ClusterIP	Prod traffic via cloud providers

3. When should you use ClusterIP in production?

Answer:

- For **internal microservice communication**
 - Between backend services (e.g., auth → user-db)
 - Where exposing externally isn't required
 - When used with **Ingress** or **Service Mesh**
-

4. What are the limitations of NodePort services in production?

Answer:

- Static port range (usually 30000–32767)
 - Not scalable for large applications
 - Security risk if left unfiltered
 - No load balancing across nodes unless fronted by a load balancer
-

5. What happens if a NodePort is already in use?

Answer:

The Service creation fails with a conflict error:

Error from server (AlreadyExists): services "example" already exists

Only one Service can bind to a NodePort per cluster node at a time.

6. What is an ExternalName service? When would you use it?

Answer:

It maps a Kubernetes Service to an **external DNS name**. It's used when:

- You want to access **external APIs** or **legacy systems** using internal DNS
- You want to abstract external dependencies

spec:

type: ExternalName

externalName: api.external.com

7. How does Kubernetes do service discovery internally?

Answer:

- Kubernetes assigns each Service a **ClusterIP**
 - A **DNS name** is created by **kube-dns** or **CoreDNS**
 - Pods resolve service names like **<service>.<namespace>.svc.cluster.local**
-

8. How does kube-proxy route traffic for services?

Answer:

kube-proxy runs on each node and:

- Watches for Service and Endpoint changes
 - Uses **iptables**, **IPVS**, or **userspace** mode to route Service IPs to Pod IPs
 - Ensures load balancing and high availability
-

9. How do Services handle session persistence (sticky sessions)?

Answer:

Kubernetes supports **ClientIP-based session affinity**:

spec:

sessionAffinity: ClientIP

This causes connections from the same IP to go to the same backend pod (as long as it's healthy).

10. Can a service load balance across pods in different namespaces?

Answer:

No. Services are **namespace-scoped** and only target pods within the same namespace. For cross-namespace access:

- Use the **full DNS name** (`service.namespace.svc.cluster.local`)
 - Or expose it via **Ingress** or **Gateway**
-

11. How do health checks affect Service availability?

Answer:

- Kubernetes Services rely on **Endpoints**.
 - If a pod fails readiness probes, it's **removed from Endpoints**, and traffic is not routed to it.
 - Ensures only healthy pods receive traffic.
-

12. What's the difference between **targetPort**, **port**, and **nodePort** in a Service?

Answer:

Field	Description
targetPort	Port the container listens on
port	Port exposed by the Service (ClusterIP)
nodePort	Port on each node (only in NodePort/LoadBalancer)

13. Can you have multiple ports in a Service?

Answer:

Yes. A Service can expose multiple ports:

ports:

- name: http
port: 80
targetPort: 8080
 - name: metrics
port: 9090
targetPort: 9091
-

14. How do LoadBalancer services work with cloud providers?

Answer:

- Kubernetes provisions a **cloud-native load balancer** (e.g., AWS ELB, GCP LB).
- The load balancer routes external traffic to the **NodePort** on nodes.
- This requires cloud-controller-manager to be integrated.

15. What are best practices for using services in production?

Answer:

- Prefer **ClusterIP** + **Ingress** or service mesh (e.g., Istio)
- Avoid exposing services via **NodePort** unless necessary
- Use **readiness probes** for traffic health
- Apply **NetworkPolicies** to restrict traffic
- Monitor **Endpoints** to detect unhealthy services
- Use **Headless Services** for stateful workloads (e.g., databases)

16. How can you expose an internal service for debugging without creating a LoadBalancer?

Answer:

Use **port-forwarding**:

```
kubectrl port-forward svc/my-service 8080:80
```

This allows local access to a ClusterIP service from your workstation.

17. What are headless services and when are they used?

Answer:

Headless services (**clusterIP: None**) don't assign a ClusterIP. DNS queries return **pod IPs** directly.

Use cases:

- StatefulSets
 - Direct client-to-pod communication (e.g., for databases)
 - Service discovery for non-load balanced apps
-

18. How do you test DNS resolution for services in a pod?

Answer:

Run a DNS test inside a pod:

```
kubectl exec -it <pod> -- nslookup <service>.<namespace>.svc.cluster.local
```

19. Can a Service point to external IPs directly?

Answer:

Yes, using the `externalIPs` field:

spec:

externalIPs:

- 192.168.1.100

Kubernetes routes traffic from that IP to the Service backend.

20. How can you debug service-to-pod connectivity issues?

Checklist:

- Is the **Service and Endpoint** created?

- Is the pod passing **readiness probes**?
 - Use `kubectl get ep <service>` to see endpoints
 - Use `kubectl exec + curl` or `nslookup` inside a pod
 - Check **NetworkPolicies**, firewalls, or CNI issues
-